

JavaScript Notes — Day 5

Chapter 5: Functions and Higher-Order Array Methods

What is a Function?

A block of code that performs a specific task, which can be reused by calling it whenever needed.

Basic Function (no parameters)

```
function hello() {  
  console.log("i am chiranth");  
}  
  
hello(); // call the function to run it
```

Function with Parameters

```
function myFunction(msg) {  
  console.log(msg);  
}  
  
myFunction("i love JS");
```

Function that Calculates a Sum

```
function sum(a, b) {  
  let add = a + b;  
  console.log(add); // could also use "return add;" instead  
}  
  
sum(4, 19); // 23
```

Note: always declare a new variable with let/const (let add = a + b). Without it, JavaScript looks for an existing global variable named add, which can cause errors. Also, variables declared inside a function are **local variables** — they only exist within that function.

Arrow Functions

A shorter syntax for writing functions, using =>. Commonly stored in a const.

```
const s = (a, b) => {  
  console.log(a + b);  
};  
  
s(3, 4); // 7
```

Practice — Count Vowels in a String

Regular function version:

```
function countVowels(str) {
  let count = 0;
  for (const char of str) {
    if (
      char === "a" || char === "e" || char === "i" ||
      char === "o" || char === "u"
    ) {
      count++;
    }
  }
  console.log(count);
}

countVowels("abcdef"); // 2
```

Same problem as an arrow function:

```
const countVowels = (str) => {
  let count = 0;
  for (const char of str) {
    if (
      char === "a" || char === "e" || char === "i" ||
      char === "o" || char === "u"
    ) {
      count++;
    }
  }
  console.log(count);
};
```

forEach Loop

Runs a callback function once for every element in an array. This is a Higher-Order Function (HOF) — a function that takes another function as an argument.

Syntax: `arr.forEach(callbackFunction);`

```
let arr = [1, 2, 3, 4, 5];
arr.forEach(function printValue(val) {
  console.log(val);
});
```

Note: writing `[ ,2,3,4,5]` leaves a hole (empty slot) at index 0 — use `[1,2,3,4,5]` instead.

Practice — Print Squares Using forEach

```
let arr = [1, 2, 3, 4, 5, 6];

// method 1: inline function
arr.forEach(function square(num) {
  console.log(num * num);
});
```

```
// method 2: named function passed by reference
function square(num) {
  console.log(num * num);
}
arr.forEach(square);
```

Other Higher-Order Array Methods

Method	Purpose	Returns
map()	Transforms every element, builds a new array	New array (same length)
filter()	Keeps only elements that pass a test	New array (subset)
reduce()	Combines all elements into a single value	A single value

map()

Use map's callback return value to build a new array — don't just console.log inside it.

```
let a = [1, 2, 345, 5, 6];

let newArr = a.map(function num(val) {
  return val; // return builds the new array
});
console.log(newArr);
```

filter()

The callback returns true/false — only elements where it returns true are kept, and you must store the result.

```
let a = [1, 2, 3, 4, 5, 6, 7];

let evens = a.filter((val) => {
  return val % 2 == 0;
});
console.log(evens); // [2, 4, 6]
```

reduce()

Reduces the whole array to one value by repeatedly combining an accumulator (res) with each element (curr).

```
let a = [1, 2, 3, 4];
const n = a.reduce(function fil(res, curr) {
  return res + curr;
});
console.log(n); // 10 (sum of all elements)
```

reduce() — Finding the Largest Number

```
let a = [1, 2, 3, 4];
const n = a.reduce(function fil(res, curr) {
```

```
    return res > curr ? res : curr;
  });
  console.log(n); // 4
```

Practice Problems

1. Filter marks above 90

```
let marks = [90, 99, 76, 87, 98, 91, 86];

let topper = marks.filter((val) => {
  return val > 90;
});

console.log(topper); // [99, 98, 91]
```

2. Build an array from user input, then sum it with reduce

```
let n = prompt("enter a number :");
let a = [];

for (let i = 1; i <= n; i++) {
  a[i - 1] = i;
}
console.log(a);

let sum = a.reduce((res, curr) => {
  return res + curr;
});
console.log(sum);
```

Note: loop condition should be `i <= n` (`n` from `prompt()` is a string, so comparisons still work loosely but `Number(n)` is safer). Also print the same variable you calculated — `sum`, not `summ` or `s`.

3. Factorial of a number

```
function factorial(n) {
  let fact = 1;
  for (let i = 1; i <= n; i++) {
    fact *= i;
  }
  console.log(fact);
}

factorial(5); // 120
```